

¿Por qué es importante conocer en detalle, las necesidades del cliente en el momento de desarrollar un software?

R//

Es importante para que el software sea realmente útil para y facilite la gestión y procesos de la empresa por medio de este.

Definir la arquitectura y tecnología correctas:

Existe una regla en ingeniería de software que dice que arreglar un error o cambiar un requerimiento durante la fase de diseño es barato, pero hacerlo cuando el código ya está en producción cuesta hasta 100 veces más.

Reducción drástica de costos y tiempos:

Existe una regla en ingeniería de software que dice que arreglar un error o cambiar un requerimiento durante la fase de diseño es barato, pero hacerlo cuando el código ya está en producción cuesta hasta 100 veces más.

Facilita las pruebas:

Solo puedes comprobar que un software funciona correctamente si sabes con exactitud qué se supone que debe hacer.

En el momento de recolectar la información para el proyecto software, el cliente no suministró toda la información suficiente. ¿Considera(n) qué esto puede afectar en algo el desarrollo del proyecto? Justifique(n) su respuesta.

R//

Cuando los requerimientos son vagos, el proyecto tiende a crecer sin control.

El cliente empieza a pedir "pequeños cambios" que en realidad alteran toda la lógica del sistema. Entender los detalles desde el día uno te permite definir fronteras claras de qué incluye el proyecto y qué queda para una versión futura, protegiendo tu tiempo y tu trabajo.

¿Qué recomendaciones tiene(n) para lograr una comunicación eficaz con el cliente y el equipo de trabajo?

R//

Con el Cliente:

Evitar tecnicismos como "estamos configurando los endpoints en FastAPI". Es mejor decir: "Estamos preparando el sistema para que la información de los pedidos viaje rápido y de forma segura". Al cliente le interesa el **resultado**, no el proceso técnico.

La técnica del "Espejo": Cuando el cliente te pida algo, repítelo con tus propias palabras: *"Entonces, lo que necesitas es que el reporte de lavandería se genere automáticamente cada lunes a las 8:00 AM, ¿es correcto?"*. Esto evita que construyas algo que nadie pidió.

Muestra avances visuales pronto y seguido: Un cliente se comunica mejor cuando ve algo tangible. En lugar de explicar la base de datos, muestra un prototipo de la pantalla o un PDF de prueba. Esto genera confianza y permite corregir el rumbo antes de que sea tarde.

Gestiona las expectativas : Si una funcionalidad es muy compleja o tomará más tiempo de lo previsto, dilo de inmediato. Es mejor una verdad incómoda hoy que una entrega fallida mañana.

Con el Equipo de Trabajo:

Documentación viva (README y Comentarios): Tu mejor forma de comunicarte con otros programadores es a través de tu código. Mantén el archivo README.md de tu repositorio actualizado con las instrucciones de instalación y explica el "porqué" de las funciones complejas en los comentarios.

Estandariza los mensajes de Git: En lugar de poner "cambios" o "fix", usa un formato claro. Por ejemplo: agrega exportación a Excel en módulo de reportes. Esto permite que cualquier compañero sepa qué hiciste sin tener que preguntarte.

Reuniones cortas y con propósito: Si trabajas en equipo, una charla de 10-15 minutos al día es suficiente. Responde tres cosas:

¿Qué hice ayer?

¿Qué voy a hacer hoy?

¿Tengo algún bloqueo (algo que no me deja avanzar)?